Stacks and Queues

Data Structures and Algorithms

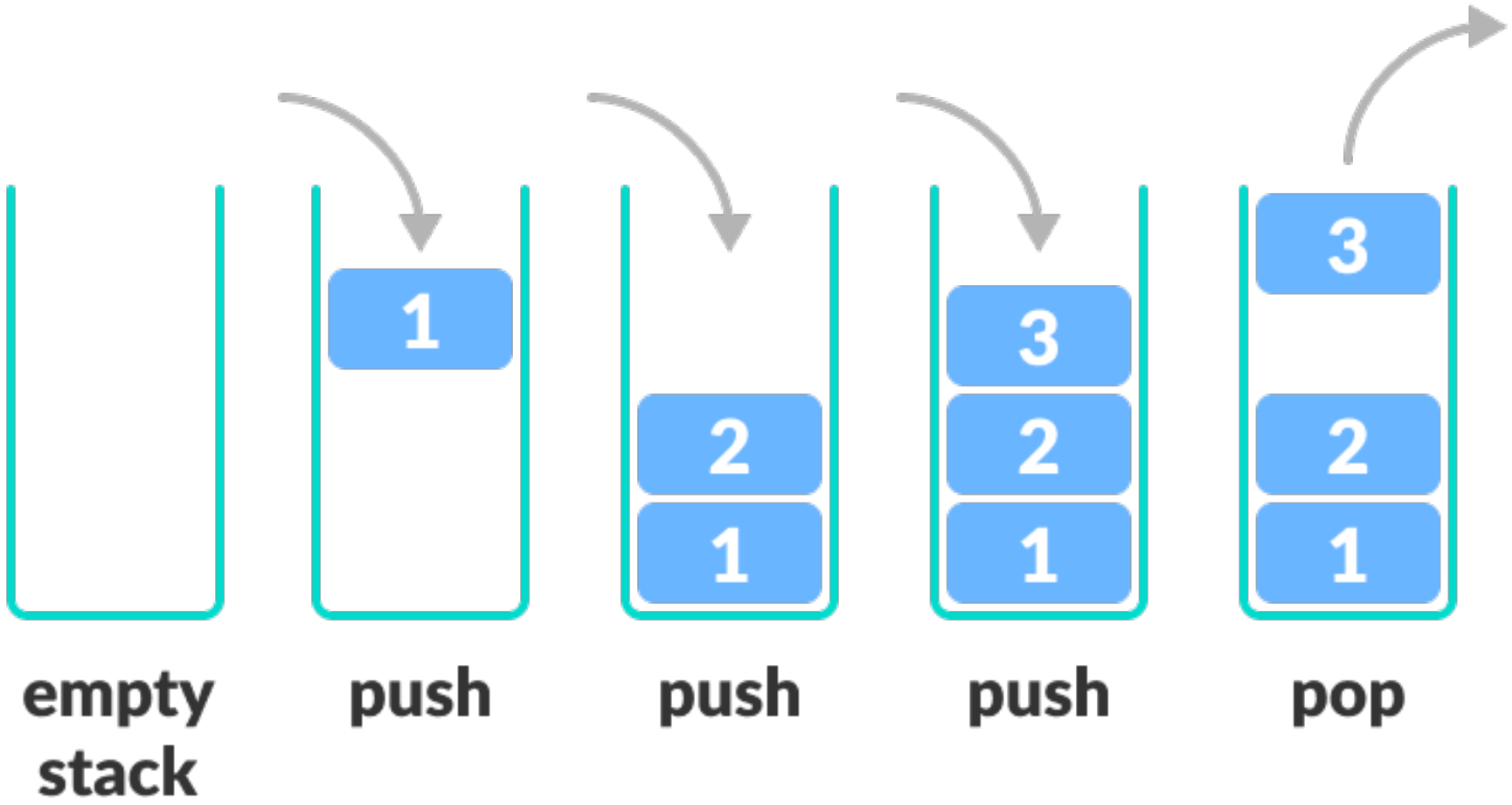
Introduction to Stacks

- A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where the last element added is the first one to be removed.
- Stacks are particularly useful for scenarios like backtracking in algorithms, such as undo operations in text editors, browser history, and parsing expressions in compilers.

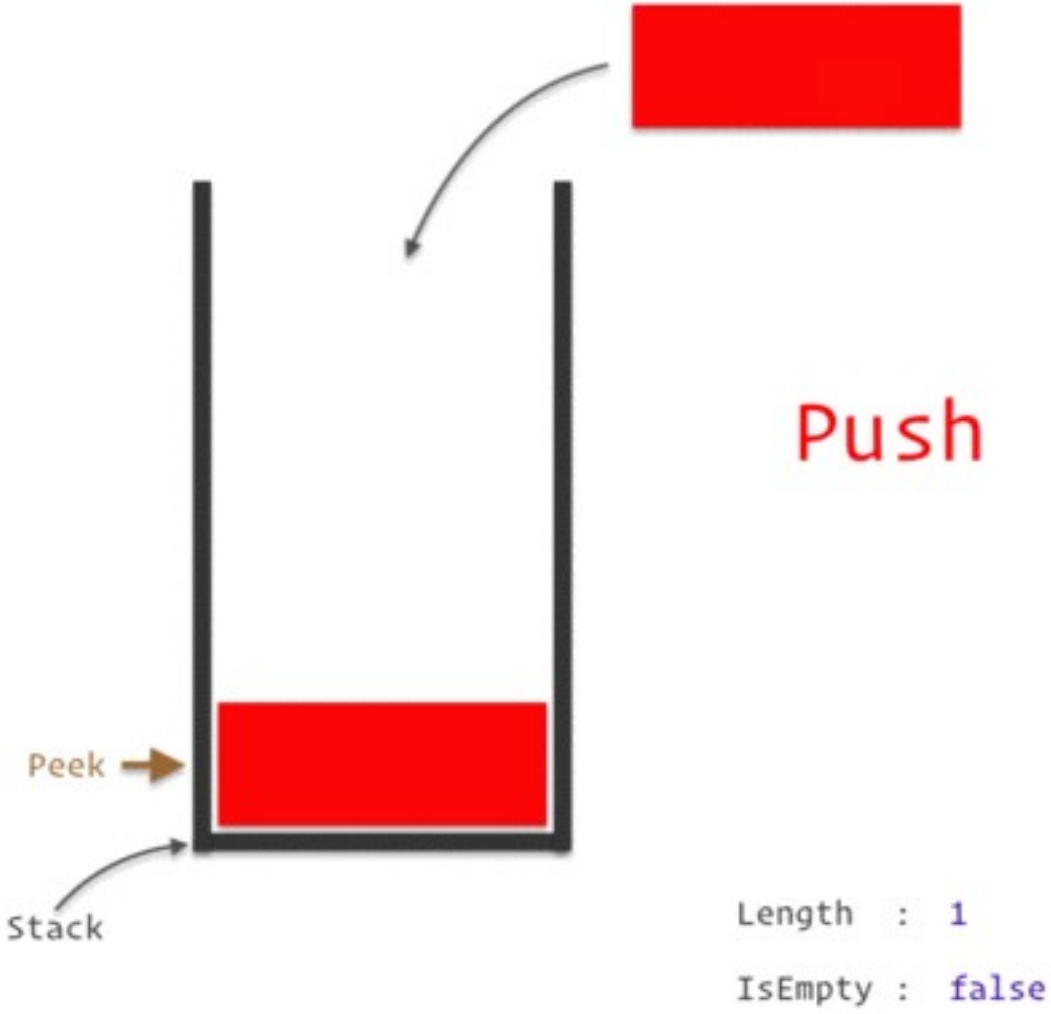
Stack Operations

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes the top element.
- **Peek:** Returns the top element without removing it.
- **isEmpty:** Checks if the stack is empty.

Stack Operations



Stack Operations



Implementing a Stack in Python

```
class Stack:
    def __init__(self):
        # Initializes an empty list to hold stack elements
        self.items = []
    def is_empty(self):
        # Returns True if the stack is empty; otherwise, False
        return len(self.items) == 0
    def push(self, item):
        # Adds item to the end of the list (top of the stack)
        self.items.append(item)
    def pop(self):
        # Removes and returns the last item (top of the stack), if not empty
        return self.items.pop() if not self.is_empty() else None
    def peek(self):
        # Returns the last item without removing it, if not empty
        return self.items[-1] if not self.is_empty() else None
    def size(self):
        # Returns the number of items in the stack
        return len(self.items)
```

Explanation

Class Definition:

35
17 The `Stack` class contains methods to perform common stack operations like `push`, `pop`, `peek`, and `size`.

`__init__` Method:

35
17 Initializes the stack by creating an empty list, `self.items`, to hold the stack elements.

`is_empty` Method:

35
17 Checks if the stack is empty. If `self.items` has a length of 0, it returns `True`; otherwise, it returns `False`.

`push` Method:

35
17 Adds a new `item` to the top of the stack by appending it to the end of `self.items`.

`pop` Method:

35
17 Removes and returns the top item of the stack (the last element in `self.items`). If the stack is empty, it returns `None`.

`peek` Method:

35
17 Returns the top item of the stack without removing it. If the stack is empty, it returns `None`.

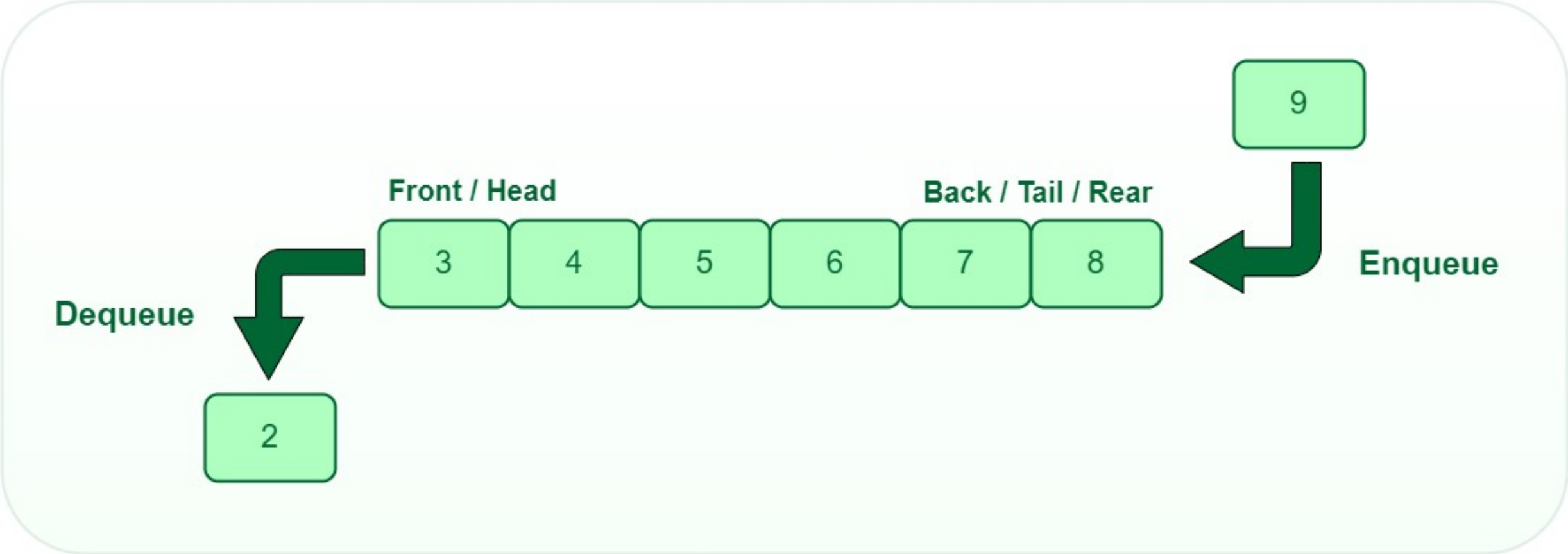
`size` Method:

35
17 Returns the number of items currently in the stack by returning the length of `self.items`.

Introduction to Queues

- A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where the first element added is the first one to be removed. Queues are commonly used in situations like task scheduling, printing jobs, and data buffering.
- Queue Operations
  - **Enqueue:** Adds an element to the end of the queue.
  - **Dequeue:** Removes the element from the front.
  - **isEmpty:** Checks if the queue is empty.

Operations of Queue



Implementing a Queue in Python

```
class Queue:
    def __init__(self):
        # Initializes an empty list to hold queue elements
        self.items = []
    def is_empty(self):
        # Returns True if the queue is empty; otherwise, False
        return len(self.items) == 0
    def enqueue(self, item):
        # Adds an item to the end of the list (end of the queue)
        self.items.append(item)
    def dequeue(self):
        # Removes and returns the first item (front of the queue), if not empty
        return self.items.pop(0) if not self.is_empty() else None
    def size(self):
        # Returns the number of items in the queue
        return len(self.items)
```

Explanation

Class Definition:

35
17 The `Queue` class provides methods to perform typical queue operations, such as `enqueue`, `dequeue`, `is_empty`, and `size`.

`__init__` Method:

35
17 Initializes the queue by creating an empty list, `self.items`, to store the elements of the queue.

`is_empty` Method:

35
17 Checks if the queue is empty by evaluating the length of `self.items`. If it has a length of 0, it returns `True`; otherwise, it returns `False`.

`enqueue` Method:

35
17 Adds an `item` to the end of the queue by appending it to `self.items`.

`dequeue` Method:

35
17 Removes and returns the first item in the queue (the item at index 0 in `self.items`). If the queue is empty, it returns `None`.

`size` Method:

35
17 Returns the current number of items in the queue by returning the length of `self.items`.

Applications of Stacks and Queues

- **Applications of Stacks:**
  - **Expression Evaluation:** Parsing expressions in postfix notation.
  - **Reversing a String:** Using a stack to reverse the order of characters in a string.
  - **Backtracking:** Algorithms that explore all possibilities, like solving a maze, can use a stack to keep track of paths.

Example of reversing a string using a stack:

```
def reverse_string(string):
    stack = Stack()
    for char in string:
        stack.push(char)

    reversed_str = ""
    while not stack.is_empty():
        reversed_str += stack.pop()
    return reversed_str
```

Explanation

Initialize a Stack:

35
17 A new `Stack` object is created to hold each character in the string.

Push Characters onto the Stack:

35
17 The function iterates over each character in the input `string` and pushes each character onto the stack. This means that the last character of the string will be at the top of the stack.

Pop Characters from the Stack:

35
17 The function initializes an empty string, `reversed_str`, to store the reversed version of the input string.

35
17 It then enters a loop that continues until the stack is empty. In each iteration, it pops the top character from the stack (which is the last character of the input string due to the LIFO nature of the stack) and appends it to `reversed_str`.

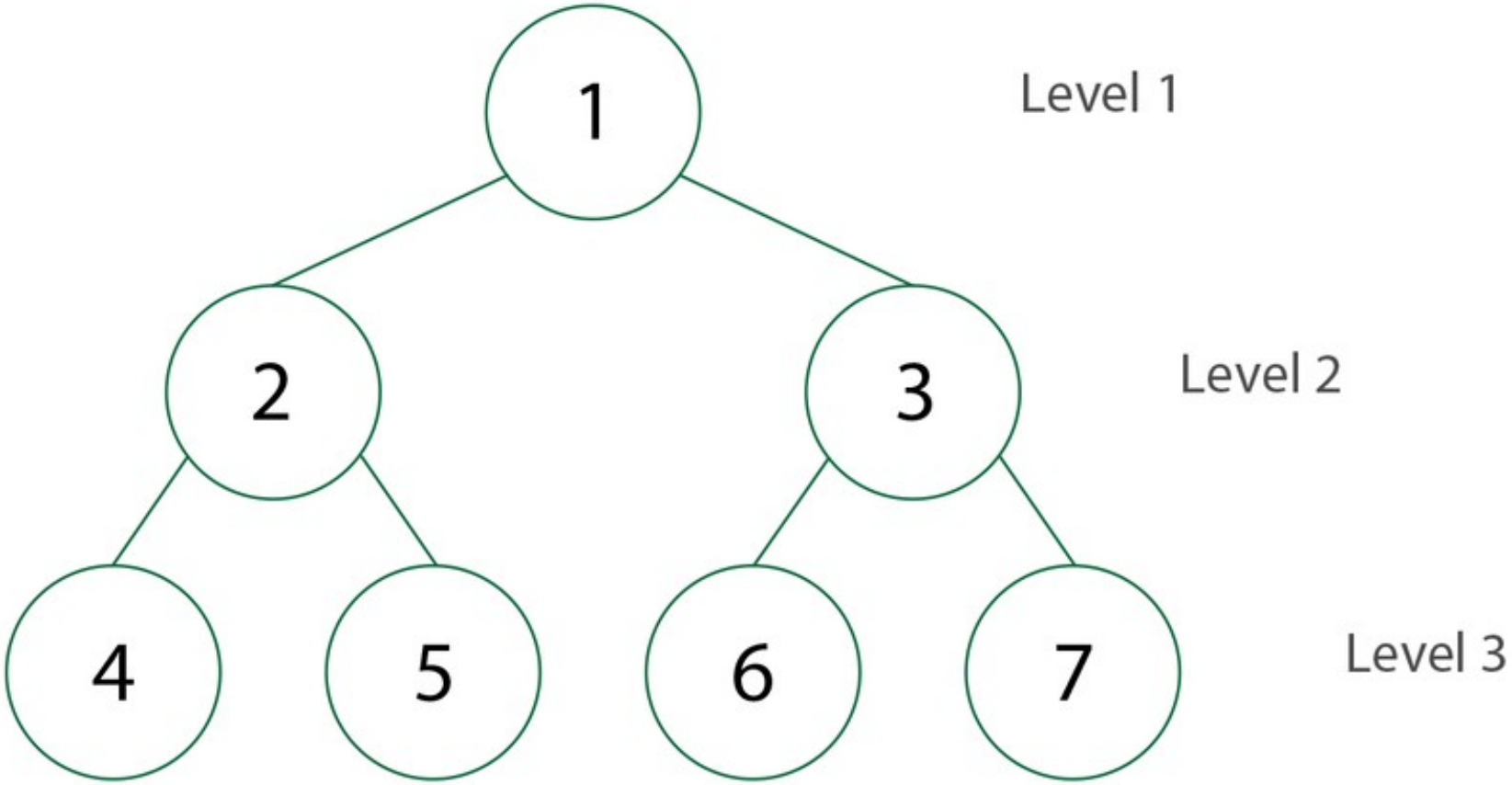
Return the Reversed String:

35
17 After all characters have been popped and added to `reversed_str`, the function returns `reversed_str`, which now contains the characters of the original string in reverse order.

Applications of Queues:

- Breadth-First Search (BFS): Used in graph traversal to explore nodes layer by layer.
- Task Scheduling: Jobs like printing tasks or OS process scheduling can use queues to ensure FIFO processing.
- Customer Service: Simulating waiting lines where the first person in line is served first.

Breadth-First Search (BFS)



Example of Breadth-First Search (BFS) using a queue:

```
def bfs(graph, start_node):
    visited = set()
    queue = Queue()
    queue.enqueue(start_node)
    visited.add(start_node)

    while not queue.is_empty():
        node = queue.dequeue()
        print(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.enqueue(neighbor)
                visited.add(neighbor)
```

Explanation

Initialize `visited` Set and `Queue`:

35
17 A `visited` set is created to keep track of nodes that have already been visited, ensuring each node is only processed once.

35
17 A `Queue` object is initialized to manage the nodes to be explored next.

Enqueue the Start Node:

35
17 The `start_node` is added to the `visited` set and enqueued in the `queue`, marking it as the starting point for the search.

BFS Loop:

35
17 The loop continues until the queue is empty.

35
17 In each iteration:

- The node at the front of the queue is dequeued (processed).
- The node is printed, representing its visitation.
- For each neighbor of the current node, the function checks if it has been visited:
  - If not, the neighbor is enqueued, and it's added to the `visited` set to prevent reprocessing.